

Assignment 3 | Bridge Pattern

Student: Bakdaulet Begaliyev

Group: 2.106

Topic: D - Remote controls

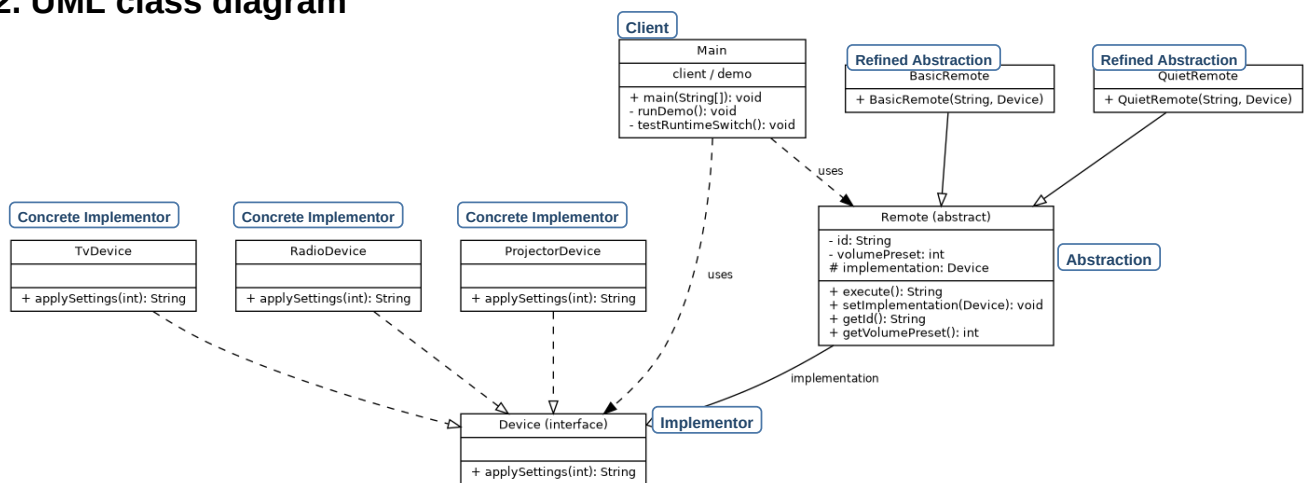
Course: ShP-2216 Software Design Patterns

1. Idea and why Bridge fits

The application has two things that can change independently. The first dimension is the type of remote: BasicRemote or QuietRemote. The second dimension is the device: TvDevice, RadioDevice, or ProjectorDevice. Bridge fits because Remote keeps a reference to the Device interface instead of depending on a concrete device class.

BasicRemote uses volume 30. QuietRemote uses volume 5. The remote sends this value to the selected Device by calling applySettings(). Because the connection is composition, one Remote object can replace its Device at runtime.

2. UML class diagram



The field **implementation: Device** is the bridge between the two hierarchies. Main is only the client that creates objects and runs the seven checks.

3. Five Clean Code examples

Separation of responsibilities

```
public String execute() {  
    return implementation.applySettings(volumePreset);  
}
```

Remote decides what preset to use, while Device decides how a concrete device applies it. The classes do not mix both responsibilities.

Meaningful names

```
public void setImplementation(Device implementation) {  
    this.implementation = implementation;  
}
```

The method name describes exactly what is changed. The parameter type also shows that any Device implementation can be used.

Small focused method

```
public String applySettings(int volume) {  
    return "TV | power=ON | volume=" + volume;  
}
```

TvDevice has one small operation. It only produces the TV state required by this assignment.

Avoiding duplicated test workflow

```
private static void testResult(String id, String classes,  
    Remote remote, String expected) {  
    String actual = remote.execute();  
    boolean ok = actual.equals(expected);  
    // print PASS or FAIL  
}
```

T1-T4 and T6-T7 reuse one helper instead of repeating the same comparison and output code in Main.

Encapsulation of state

```
private final String id;  
private final int volumePreset;  
protected Device implementation;  
  
public String getId() { return id; }  
public int getVolumePreset() { return volumePreset; }
```

The ID and preset are stored inside Remote. They cannot be changed directly from Main, which helps prove in T5 that domain state stays unchanged.

4. Runtime switch (T5)

T5 creates one BasicRemote with TvDevice, saves a second reference to the same object, executes it, calls setImplementation(new RadioDevice()), and executes again. The expression originalReference == remote checks reference equality. The ID and volume preset are also compared before and after the switch. The result changes from TV to RADIO, but the Remote object and its domain data stay the same.

5. Independent extension with I3

The first committed version contained Device, TvDevice, RadioDevice, Remote, BasicRemote, QuietRemote, and T1-T5. After that, ProjectorDevice was added as I3. The existing abstraction classes, Device interface, TvDevice, and RadioDevice were not changed. Only the new ProjectorDevice and Main are changed inside src in extension.diff. This demonstrates that a new implementation can be added independently.

A specific trade-off of Bridge is that there are more classes and more indirection than in a very small program with direct calls. For this assignment that extra structure is acceptable because the two dimensions are clearly separated.

6. Bridge compared with Adapter

Bridge is chosen before or during the design to separate abstractions and implementations that should vary independently. Adapter has another intent: it wraps an existing class so that it can be used through a different interface expected by a client. In this project there is no incompatible old API to convert, so Bridge is the relevant pattern.

7. Demonstration result

```
T1 PASS | BasicRemote + TvDevice | result=TV | power=ON | volume=30
T2 PASS | BasicRemote + RadioDevice | result=RADIO | power=ON | volume=30
T3 PASS | QuietRemote + TvDevice | result=TV | power=ON | volume=5
T4 PASS | QuietRemote + RadioDevice | result=RADIO | power=ON | volume=5
T5 PASS | BasicRemote runtime switch | sameObject=true | stateUnchanged=true
T6 PASS | BasicRemote + ProjectorDevice | result=PROJECTOR | power=ON | volume=30
T7 PASS | QuietRemote + ProjectorDevice | result=PROJECTOR | power=ON | volume=5
SUMMARY: 7/7 PASS
```

8. References

1. Assignment 3 | Bridge Pattern, Astana IT University, ShP-2216 Software Design Patterns, 2026-2027.
2. Refactoring.Guru. "Bridge." <https://refactoring.guru/design-patterns/bridge>
3. Refactoring.Guru. "Adapter." <https://refactoring.guru/design-patterns/adapter>